



Final 2024

Lösningförslag

Jage

Av: Olle Lapidus

Lösningssidé

Håll koll på vilka som är “giltiga” jagare just nu.

Använd sets/dictionaries för att det ska gå snabbt.

Lösningssidé

För varje event “alfa tar beta” i kronologisk ordning:

- Lägg till beta i mängden med jagare

- Om alfa inte är jagare:

 - Lägg till alfa i mängden med fuskare

- Om alfa är jagare:

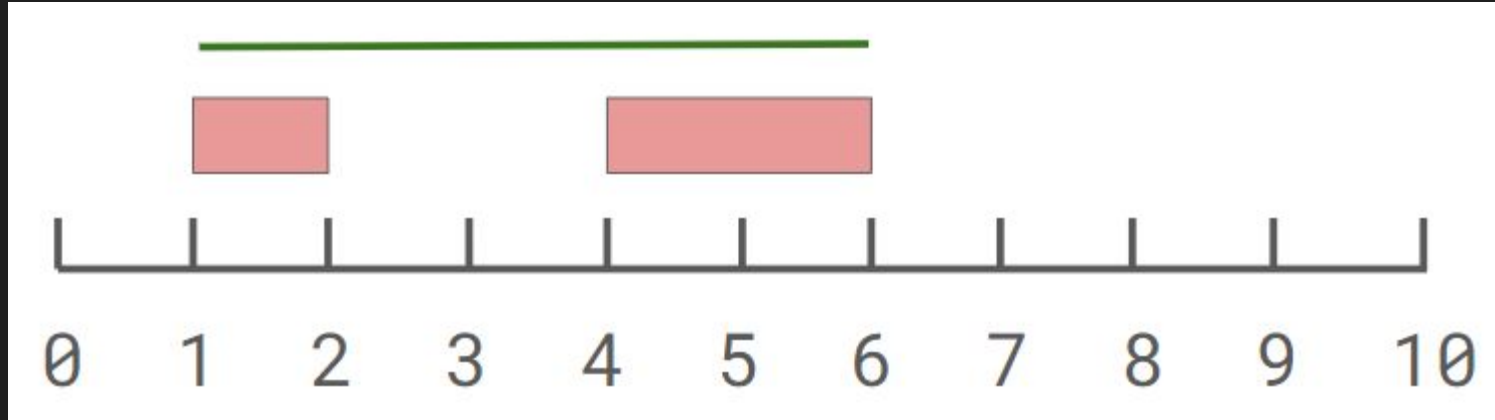
 - Ta bort alfa från mängden med jagare

Vasaloppet

Av: Nils Gustafsson

Problem

Givet N disjunkta (icke-överlappande) intervall av reklam på en tallinje,
vad är det minsta mängden icke-reklam vi kan ha ifall vi väljer ett intervall på storlek S ?



Problem

NAMN	GENOMSnittlig POÄNG	GENOMSnittligt ANTAL FÖRSÖK	GENOMSnittligt ANTAL FÖRSÖK FÖR FULL POÄNG
A Tag	3360/61 (55)	1,69	1,69
B Vasaloppet	1465/113 (13)	3,90	3,71
C Floor Area	795/29 (27)	2,42	2,11
D Avalanche	625/10 (62)	1,11	1,00
E The Dark Chambers of Chalmers	0/0 (0)	--	--
F The Ice Puzzle	40/2 (20)	2,00	2,00
G Trafikverket's Mistake	0/0 (0)	--	--

Statistik 1 timme och 40 minuter in i tävlingen.

Subtask 1

(10 p, $N = 1$) $O(N) = O(1)$

Det finns bara 1 reklam, så vi vill alltid försöka ha med det i intervallet.

```
n,t,s = map(int,input().split())

for _ in range(n):
    l,r = map(int,input().split())
    print(max(0,s-(r-l)))
```

$O(N)$ -lösning

```
n,t,s = map(int,input().split())

l,r = map(int,input().split())
print(max(0,s-(r-l)))
```

$O(1)$ -lösning

Subtask 2

(25 p, $N \leq 1000$)

Ifall $N == 0$:

Då kommer vi alltid behöva missa s sekunder.

annars:

Vårt intervall på storlek s kommer alltid vilja börja antingen där ett reklam börjar/slutar. Vi testar att börja på alla reklam-start och reklam-slut, och kollar hur itererar igenom alla andra reklam tills vi har ett intervall på storlek s . (m.h.a. lite matte) $\rightarrow O(N^2)$

Subtask 3

(30 p, $T \leq 10^6$)

Vi kan skapa en lista med 10^6+1 nollor.

Varje index representerar 1 sekund i listan. Ifall vi har ett reklam som börjar vid sekund L och slutar vid sekund R , så sätter vi alla element mellan index L och $R-1$ till 1. $O(T)$ eftersom alla reklam är disjunkta.

Sedan kan vi antingen använda en prefix-summa alt två-pekare-metoden för att kolla alla intervall med storlek s .

→ $O(T)$

Fullösning

Vi kan kombinera allting vi har använt i de föregående subtasken:

Lösning 1:

Vi kan skapa en lista med alla start och slutpunkter på alla reklam. Vi ser även till att listan är sorterad så att alla talen är i ökande ordning. Sedan kan man använda 2-pekare-metoden och testa alla intervall av storlek s som börjar på en start-/slut-punkt. $O(N)$

Ganska jobbig att implementera, dessutom måste man specialhantera ifall intervallet med storlek s slutar på en start-/slut-punkt eller mellan 2 punkter.

Fullösning

Lösning 2:

(Ifall man inte gillar två-pekare-metoden)

Vi skapar samma lista, och testar att börja intervallet vid alla start/slut, och sedan binärsöka vart intervallet ska sluta. Sen utifrån det kan man t.ex. ha en prefix-summa över alla längder av intervall och räkna ut antalet sekunder som är reklam. $O(N \log N)$

Behöver fortfarande specialhantera fallet när slutet hamnar mellan punkter eller inte.

Golvyta

Av: Nils Gustafsson

Problem

Givet en sträng S av längd $(1 \leq K \leq 3 \cdot 10^5)$ bestående av karaktärerna ' $<$ ', ' $\wedge$ ', ' $>$ ', ' v ', och två heltal $(0 \leq r, c \leq 3 \cdot 10^5)$, bestäm den minsta arean på ett rum sådant att följande gäller:

- Rummet är en rektangel med sidlängd N, M vars minsta punkt är $(0,0)$
- Om en robot börjar i $(0,0)$ och rör sig utifrån S hamnar den på (r, c)
 - Om roboten försöker åka in i en vägg händer ingenting



Lösning för sample 1

Subtask 1

- Robot rör sig endast ner och till höger
 - Fel i statement
- $\text{Svar} = (\# \text{högerpilar} + 1) * (\# \text{nedåtpilar} + 1)$

Subtask 2: $K \leq 100$

- Simulera för varje storlek av rum med $1 \leq N, M \leq K+1$
- Svaret = minsta arean då det fungerar
- $O(K^3)$

Subtask 3: $K \leq 5000$

- Insikt: höjden och bredden på rummet är oberoende av varandra
- Simulera alla $1 \leq N, M \leq K+1$ separat
- Svar = (minsta bredd som fungerar) * (minsta höjd som fungerar)
- $O(K^2)$

Full lösning

- Insikt: Slutpositionen kan aldrig minska när N eller M ökar
- Binärsök över N,M separat
 - Simulering för att avgöra om slutpositionen är mindre eller större än vad den ska vara för ett visst N eller M
- Svar = $N * M$
- $O(K \log K)$

Laviner

Av: Olle Lapidus

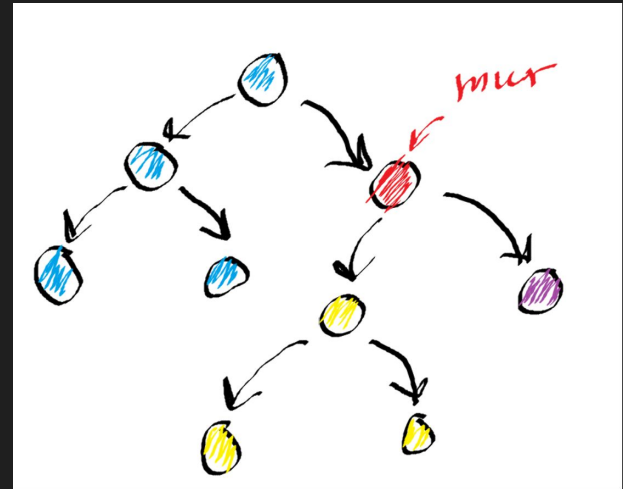
Förberett av: Olle Lapidus och Harry Zhang

Problem:

Givet ett rotat träd, (där kanter är riktade bort utåt från roten), ta bort K noder så att den största komponenten som blir kvar är så liten som möjligt.

Riktning på kanter visar sig vara obetydande – du kan alltid sätta ut snö högst upp i komponenten

Att placera en mur är ekvivalent med att ta bort noden



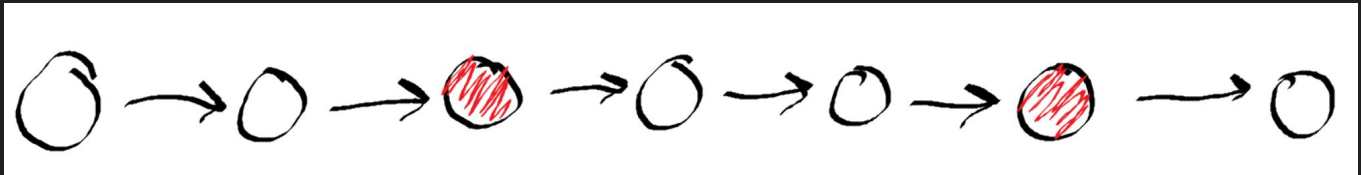
Subtask 1

Alla skidbyar ligger i en linje – det går en backe från 1 till 2, en från 2 till 3, en från 3 till 4, osv.

Lösning:

Sätt ut alla murar jämt fördelade, så att varje "komponent" av skidbyar är ungefär lika stor

$N = 7, K = 2$
ger svar 2



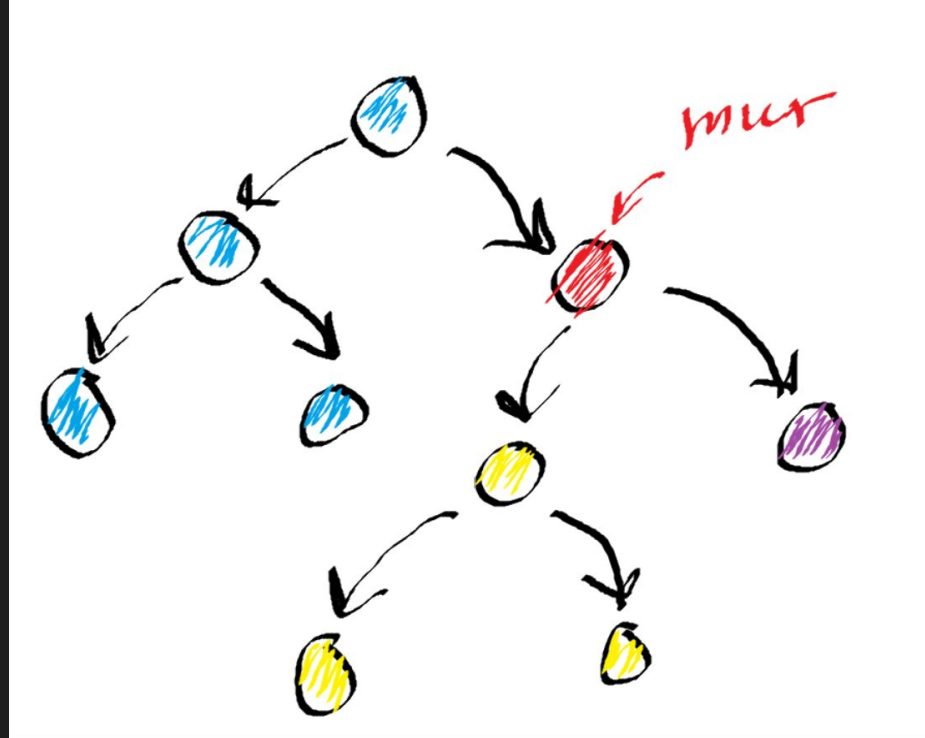
Subtask 1

Svaret kan beräknas i konstant tid med

$$\text{ceil}((N - K) / (K + 1))$$

Subtask 2

Du har bara en mur. Den kommer att dela in grafen i ett antal komponenter:



Subtask 2

Lösning: Testa alla möjliga noder att sätta ut muren i, och använd dfs/bfs/din favoritalgoritm för att hitta storleken av alla komponenter. Svaret är den största av dessa.

Med fördel, använd dfs (rekursiv), och börja dfsen i noden du murar. Ha storleken på komponent som return-värde:

```
def dfs(nod, vart jag kom ifrån):  
    storlek = 1          # jag själv  
    för varje granne som inte är den jag kom ifrån:  
        storlek += dfs(granne, nod)  
    return storlek
```


Subtask 3, 4, 5

Subtask 3: Använd bitmasks för att iterera över alla möjliga delmängder av noder att placera murar i

Subtask 4: Använd pillig träd-DP för att bestämma hur mycket storleken på komponenterna ändras (och vilka komponenter som finns) när du flyttar muren ett steg snabbt

Subtask 5: Samma idé som fullösning, men suboptimalt implementerat

Fullösning

Vi kan gissa på hur stor största komponenten kommer bli, och sedan girigt placera ut murar.

Om vi lyckas med att placera ut murarna vet vi att vår gissning är mindre än eller lika med det sanna svaret.

→ Vi kan binärsöka över svaret

Fullösning

Vi placerar ut murarna girigt med hjälp av dfs. Som returnvärde har vi både storleken på den komponenten vi är i just nu, och antalet murar vi placerat ut i subträdet till denna nod.

Fullösning

```
def dfs(nod, x = största tillåtna komponentstorlek, p = vart kom jag ifrån?):  
    storlek = 1  
    murar = 0  
    för varje granne som inte är p:  
        granne_storlek, granne_murar = dfs(granne, x, nod)  
        storlek += granne_storlek  
        murar += granne_murar  
    om storlek > x:  
        murar += 1  
        storlek = 0  
    return storlek, murar
```

Fullösning

Vi binärsöker över värdet på x , och för varje gissning kollar vi:

$\text{storlek, murar} = \text{dfs}(\text{rot}, -1, x)$

om $\text{murar} \leq K$:

 då måste svaret vara $\leq x$

om $\text{murar} > K$:

 då måste svaret vara $> x$

Chalmers mörka källare

Av: Olle Lapidus

Lösningssidé

Gå först högst upp till toppen. Du kan då nå till målet genom att bara gå ner för trappor → appen kommer ge dig “djupet” av målrummet som svar, alltså du får reda på vilken våning målet ligger på.

Gå neråt och se till att hitta vilket rum som ligger precis ovanför målet....

Subtask 1

$N = 2$

Finns väldigt få fall, kan testa alla möjliga.... Jobbigt och tråkigt.

Subtask 2

$N \leq 10$

Finns totalt $2^{11} - 1 = 2047$ rum $\rightarrow$ vi hinner besöka alla rum om vi gör det i en smart ordning. Dock måste vi vara lite kluriga med när vi gör app-queries....

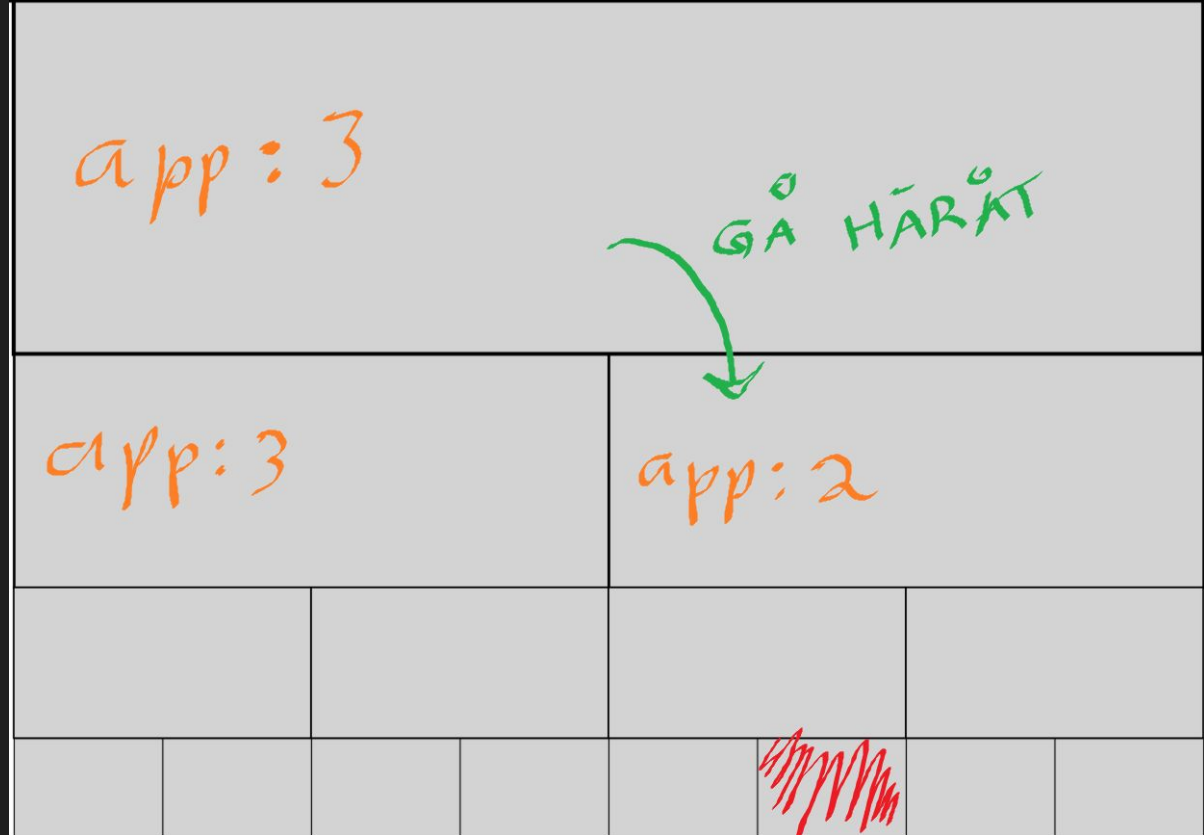
Subtask 3

$N \leq 250$

Vi har råd att göra två queries för varje våning. Jag vet att målet är under toppen. För att ta reda på om jag ska gå neråt åt vänster eller neråt åt höger frågar jag appen i båda rummen under mig och väljer det som gav minst svar.

Subtask 3

$N \leq 250$



Subtask 5

$N \leq 500$

Samma tänk som $N \leq 250$, men vi vet ju faktiskt att den ena kommer ge samma app-svar som det rummet vi är i nu, och den andra kommer ge svar - 1. Det är den som ger svar - 1 som vi ska vidare till.

Subtask 5

$N \leq 500$

app: 3							
app: 3				GÅ HÄR ÅT			

Subtask 6

$N \leq 950$

Vi vill nu försöka göra motsvarande, men bara fråga på varannan våning.

Subtask 6

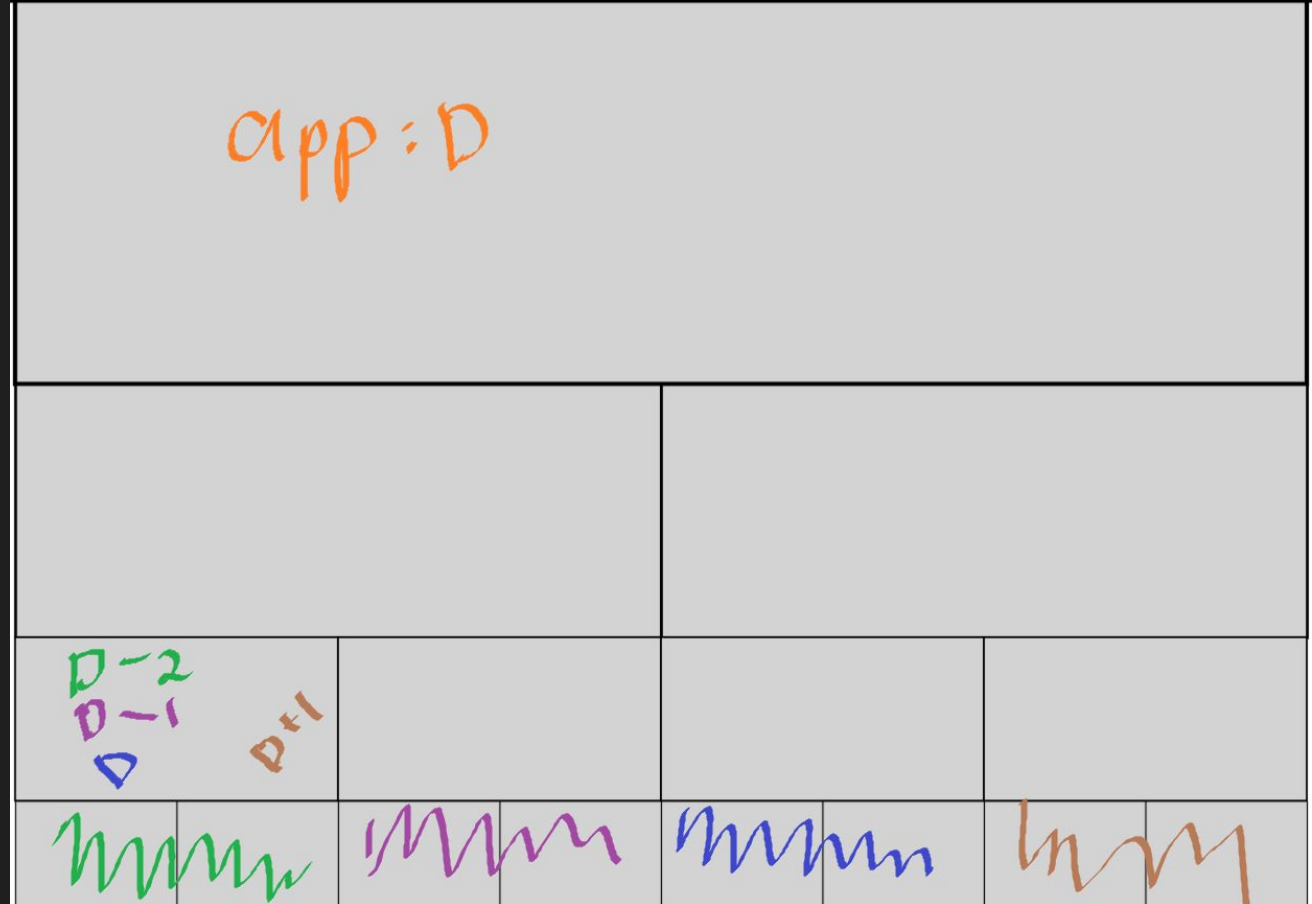
$N \leq 950$

Vi vill nu försöka göra motsvarande, men bara fråga på varannan våning.

Subtask 6

$N \leq 950$

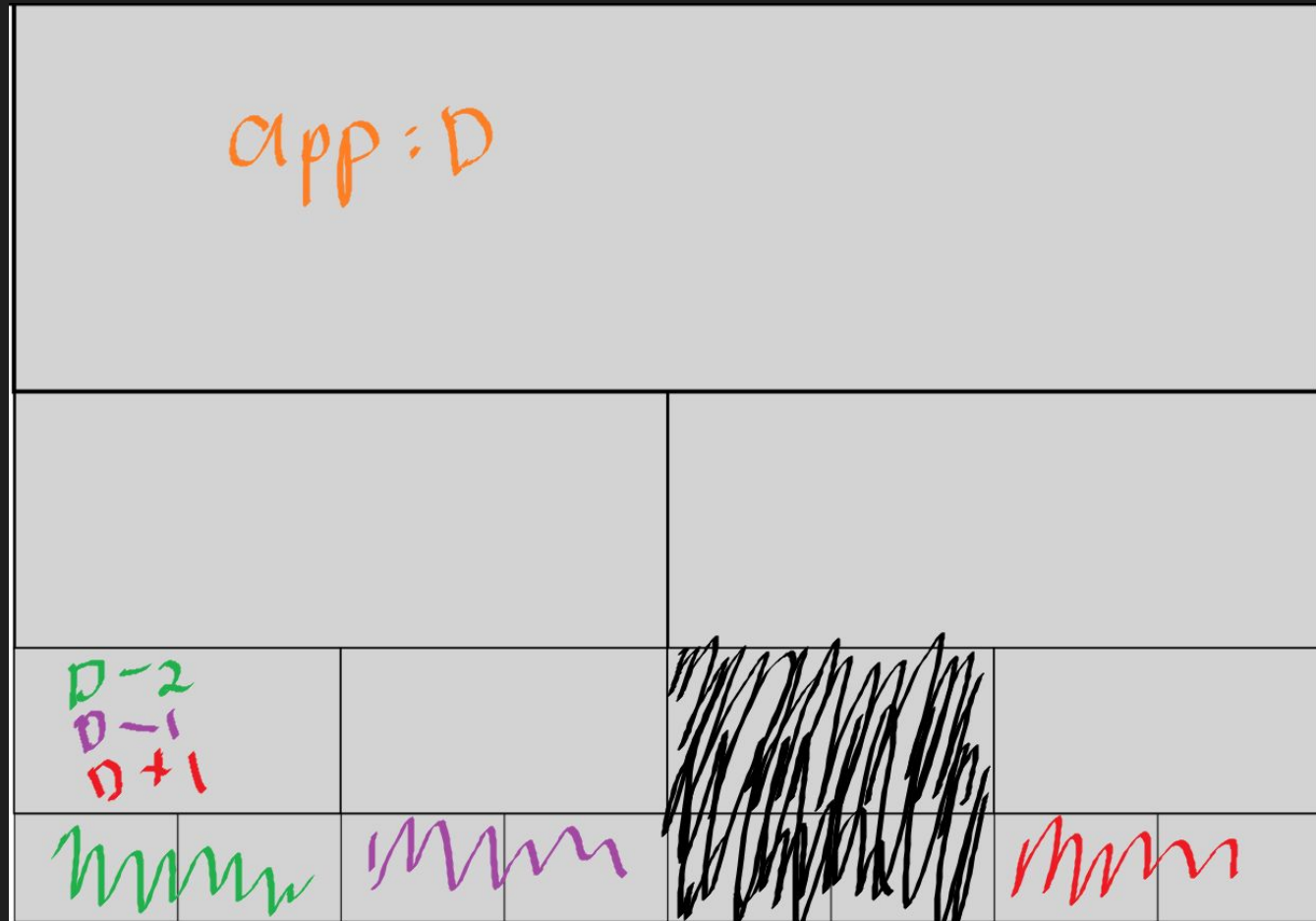
Om det inte finns
några blockade
rum:



Subtask 6

$N \leq 950$

Om det finns
blockade rum
kan vi resonera
på det här sättet:



Fullösningen

Problemet med $N \leq 950$ -lösningen är att vi nu ställer frågor på våning 1, våning 3, våning 5, ..., våning 999. När vi gjort queryn på våning 999 har vi gjort 500 (maxantal) queries, men vi har fortfarande två möjligheter för vilket som är målet....

Fullösningen

Problemet med $N \leq 950$ -lösningen är att vi nu ställer frågor på våning 1, våning 3, våning 5, ..., våning 999. När vi gjort queryn på våning 999 har vi gjort 500 (maxantal) queries, men vi har fortfarande två möjligheter för vilket som är målet....

Det visar sig att lösningen på detta är någorlunda jobbig.

Fullösningen

Vi sparar in en query i början av processen. Istället för att ställa queries enligt mönstret

$1 \rightarrow 3 \rightarrow 5 \rightarrow \dots \rightarrow 999$

vill vi göra

$4 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow \dots \rightarrow 1000$

Fullösningen

Det visar sig att för alla möjliga starter, alltså alla möjliga översta fyra våningar, går det att ställa två queries på våning 4 som ger oss tillräcklig information för att lista ut vilken av de $2^3 = 8$ rummen på våning 4 vi befinner oss under (och hur långt ner målet ligger).

Istället för att göra jobbig falluppdelening för hand kan vi fullsöka översta fyra våningarna genom att vandra upp och ner i källaren lite grann, och sedan för varje par av rum på åttonde våningen med hjälp av Djikstra testa om queries i de rummen är tillräckligt för att få information om vilket rum målet ligger under.

Fullösningen

I fallet då inga rum saknas
ser det ut så här:

Vi vet att $a + b = 7$, och får
därmed ett ekvationssystem
med två ekvationer, och två
okända

$D = ?$							
$a + D - 2$		a		$ \ 0 \ $		b	$b + D - 2$

Fullösningen

Men det finns massa olika fall och det är jättejobbigt att hårdkoda dem, så man kan låta programmet söka efter vart queries ska sättas.



Ispusslet

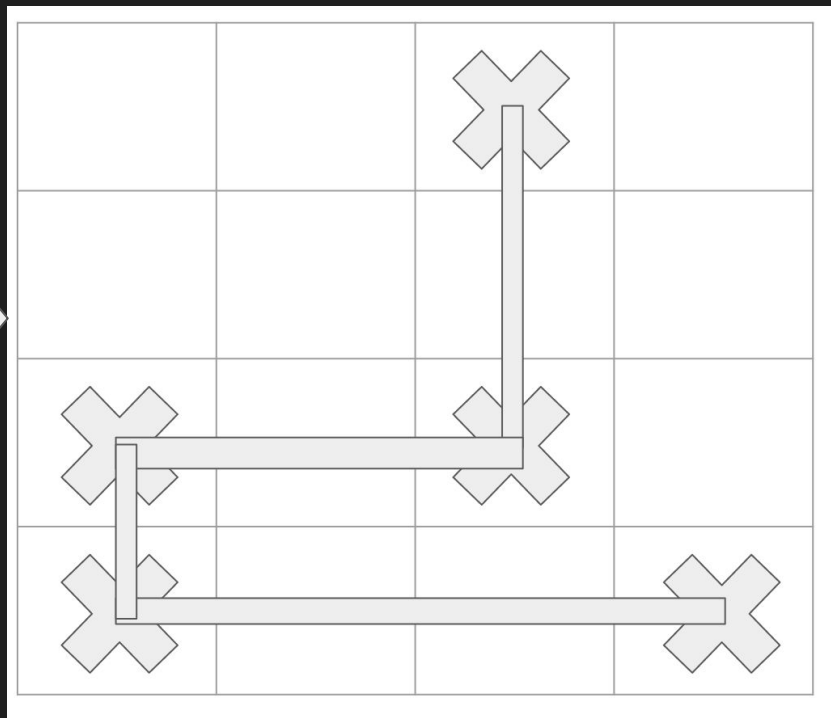
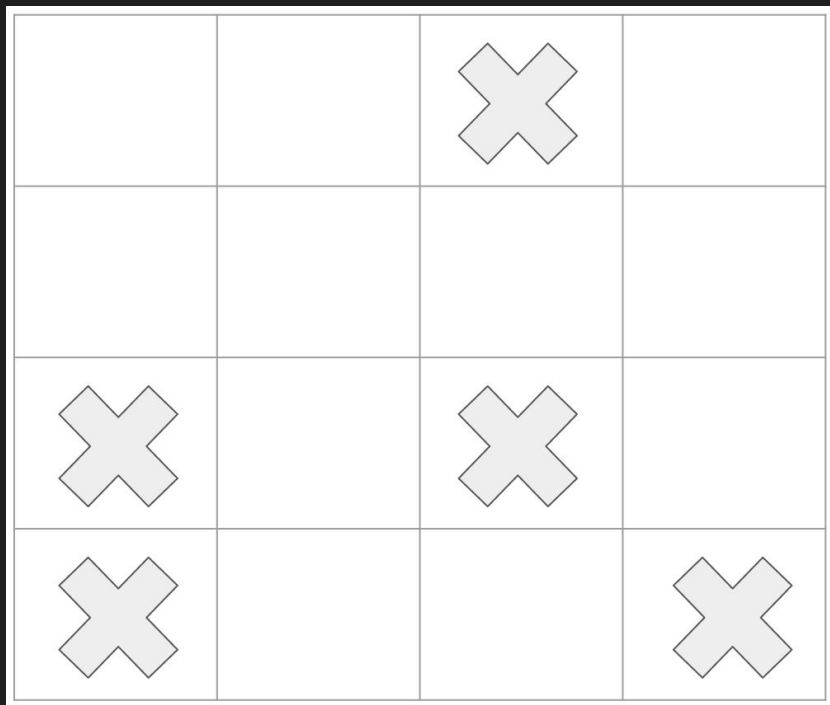
Av: Joshua Andersson

Subtask 1

Börja med att skapa följande graf: varje kryss är en nod, dra kant till alla kryss den kan nå. Går att hitta grafen i $O(nm^4)$

När är det inte möjligt? När det finns flera komponenter

Annars går det



Subtask 1

Om $r=0$ och det går, så går det också för ett godtyckligt spännande träd.

Algoritm: vandra ner till ett löv, vandra tillbaka till start. Enda förändringen är att lövet bytte färg. Detta kan ses som att vi tar bort lövet. Rinse and repeat. Om starten är 0 i slutet skippar vi sista steget.

$\sim K^2$ drag

Subtask 2 och 3

Skriv en rekursiv funktion som returnerar först när det fixat hela dess subträd. Om alla barn är fixade men inte denna noden går vi upp till föräldern och ner igen.

Naiv analys ger 4k drag. Går att bevisa att det blir 3k.

Subtask 4 och 5

Vi behöver en full karaktärisering av när det går. Observera att algoritmen i subtask 3 nästan funkade- enda sättet den kan misslyckas är att råka sätta roten av trädet till kryss i slutet.

Kan vi ändra roten till en cirkel utan att påverka resten av grafen?

IFF det finns en cykel av udda längd

Subtask 4 och 5

Om det finns en cykel av udda längd, promenera till den, gör lite grejs och vandra tillbaka. 2K för vägen, 4K för cykeln => 7k totalt (eftersom väg och cykel är disjunkta lägger vi inte ihop de).

Pill: se till du promenerar ner till noden i cykeln som är högst upp, annars det bli 9K. Behandla om roten är en del av cykeln

Annars är grafen bipartit. Om antalet noder är udda går det inte, ganska lätt att argumentera fram. Icketrivialt kommer vår trädalgoritm alltid att funka om det finns ett jämnt antal noder

Subtask 6

Optimala sättet att fixa en cykel är följande:

låt $l = \text{floor}(\text{cykelns längd}/2)$

Då är kortaste:

$\langle \langle \rangle \rangle * l + \langle$

Där $\langle$ betyder gå vänster i cykeln och $\rangle$ gå höger

Antingen kan man lösa för cykler av längd 3 (finns egentligen inte i våra grafer) och sedan betrakta hur man ska fixa att det läggs till 2 noder, eller genom att bruteforcea fram lexikografiskt minsta lösningen och se mönstret

Har längd $2k$. Totalt $2k+3k=5k$

Sammanfattning

Om $r=0$:

Lös med träd

Om $r=1$:

Funkar trädlösning?

Ja: done

annars: finns det udda cykel?

Om ja: ändra paritet och kör träd.

Om nej: garanterat udda antal noder, skriv ut -1

Trafikverkets misstag

Av: Joshua Andersson

Subtask 1

Testa alla ordningar av att skicka hem bilar. $O(C! * C * N)$

Subtask 2

Loopa igenom bilarna och kolla om det finns någon som kan köra hem. Skicka hem den. Rinse and repeat.

Worst case hittar vi en bil per iteration, varje tar $O(C*N) \rightarrow O(C^2*N)$

Subtask 3/4

C är litet-> kan vi slå bort N-faktorn? Ja

Låt $w[i]$ =noden personen arbetar på, $h[i]$ =noden de vill hem till

Låt varje nod ha vikt 0 om ingen bil står på den, vikt 1 annars.

Att en bil inte är blockerad $\longleftrightarrow$ summan av noderna i vägen mellan $w[i] \rightarrow h[i] = 0$ (specialbehandla att $w[i]$ kommer vara 1)

Så vi behöver path sum query med uppdateringar

Subtask 3

Beräkna path sum queries och lca i $O(\text{längsta vägen})$

```
int pathsum(int a, int b)
{
    int ret = active[a]+active[b];
    if (depth[a] < depth[b]) swap(a, b);
    while (depth[a] > depth[b]) a = par[a], ret += active[a];
    while (a!=b)
    {
        a = par[a];
        b = par[b];
        ret += active[a] + active[b];
    }
    return ret-active[b];
}
```

Subtask 4

Antingen kan man använda heavy-light decomposition.

Långsamt

Går att göra i $O(\log(n))$ per query/update eftersom + är inverterbart.

Skapa euler tour av trädet.

1 2 4 4 2 3 3 1

+1 -1 +1 -1 -1 +1 -1 +1

Sätt $tin[u]=1$, $tout[u]=-1$

Subtask 4

Skapa euler tour av trädet. Låt denna vara t

1 2 4 4 2 3 3 1

+1 -1 +1 -1 -1 +1 -1 +1

Sätt $tin[u]=1$, $tout[u]=-1$

Låt $sum(l,r)$ =summan av euler tour från l till r . Låt w vara start och h slutnod

Använd inklusion exklusion för att få rätt:

summan av väg mellan w och h =

$sum(0, tin[w]) + sum(0, tin[h]) - 2 * sum(0, tin[lca(w, h)]) +$
 $sum(tin[lca(w, h)], tin[lca(w, h)])$

Subtask 4

Använd inklusion exklusion för att få rätt:

summan av väg mellan w och h =

$$\text{sum}(0, \text{tin}[w]) + \text{sum}(0, \text{tin}[h]) - 2 * \text{sum}(0, \text{tin}[\text{lca}(w, h)]) + \text{sum}(\text{tin}[\text{lca}(w, h)], \text{tin}[\text{lca}(w, h)])$$

Fenwick träd kan lösa detta i $3 * \log(n)$ operationer! Extremt bra konstantfaktor

HLD funkar självklart också

Subtask 5

Betrakta följande graf: varje bil är en nod, och det finns en kant mellan den och alla bilar som blockerar dess väg. Rikta kanterna från blockerande bil till blockerad bil.

Vårt problem är ekvivalent med att hitta en topologisk sortering av den här grafen, eller avgöra att det inte finns (den är inte DAG).

Vår förra algoritm är ungefär ekvivalent med Kahns algoritm (poppa noder med ingrad 0)

Subtask 5

Vår förra algoritm är ungefär ekvivalent med Kahns algoritm (poppa noder med ingrad 0). Det känns väldigt svårt att optimera den mer...

Kan vi använda någon annan topological sorting-algoritm?
DFS-algoritmen för topologisk sortering!

Ungefär argumentet: för en nod u , så måste alla bilar som blockerar mig komma innan mig i ordning. Fortsätt så rekursivt

Subtask 5

```
vis = ...
edges = ...
order = []
def toposort(u):
    vis[u]=True
    for e in edges[u]:
        if not vis[e]:
            toposort(e)
    order.append(u)

for i in range(n):
    if not vis[i]:
        toposort(i)
```

Subtask 5

Vi kan generera kanter on-demand och sedan ta bort noden vi besöker. Vi korsar $C-1$ kanter. Totala komplexitet blir

$O(N+C*\text{get_edge}())$

Där `get_edge` är algoritmen som hittar en godtycklig bil på vägen mellan nod `a` och `b`

Subtask 5

get_edge algoritim 1:

Återanvänd fenwick-trädet+tvåpotenshopp för att binärsöka fram en nod. Går kanske med HLD+fenwickträd. $O(\log(n)^2)$, 0.9 C++

Går kanske med bottom-up segmentträd, troligtvis inte med rekursivt segmentträd

Subtask 5

get_edge algoritm 2:

För varje nod, beräkna lägsta föräldern som är en bil. Låt denna vara p .

Hitta bil: kolla om $\text{djup}[p[w]] \geq \text{djup}[\text{lca}(w,h)]$ eller $\text{djup}[p[h]] \geq \text{djup}[\text{lca}(w,h)]$

Ta bort bil: sätt $\text{active}[u]=0$. Hur uppdaterar vi förälderpekarna?

Behövs inte. Vi gör som i union-find! Med path compression blir det linjärt. Total komplexitet: $O(N+C)+O(C \text{ stycken lca-queries})$.

Går att göra i $O(N+C)$ totalt. Domarlösning: 0.3s i C++ sparse table

```
int findp(int u) { return (u == 0 || active[u]) ? u : parent[u] = findp(parent[u]); }
```

Subtask 5

Troligtvis väldigt svår att få snabbt nog i python om du inte använder annat än algoritm 2. Om du är seriös med att tävla i programmering, lär dig C++